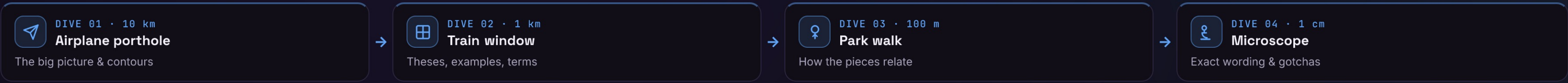




Observer

Define a one-to-many dependency so that when one object changes state, all of its dependents are notified and updated automatically.



Airplane porthole

DIVE 01 · ORIENTATION

▼ 10 km

The pattern behind event systems — it wires a one-to-many link so a subject can broadcast changes to many observers without knowing who they are.

THE CONTOURS

• One subject, many listeners
— a state change fans out to all of them

• Subject & observers stay loose
coupled only through a thin interface

• Add or drop listeners at runtime,
mind the forgotten-unsubscribe leak

USE IT WHEN — GoF applicability

A change to one object requires
changing others, and you don't know how many up front

An object should notify others
without making assumptions about who those objects are

 Behavioral family: Command · Mediator · Observer · State · Strategy — Observer broadcasts one→many directly; Mediator routes many↔many through a hub.

↓ DIVE DEEPER

Train window

DIVE 02 · KEY OBJECTS

▼ 1 km

THE THREE PARTS

Subject
keeps the observer list — attach / detach / notify

Observer
the update() interface every dependent implements

ConcreteObserver
holds a reference, reacts and syncs to the subject's state

IN THE WILD

Node EventEmitter
.on() / .emit() — the core pub-of-events API

DOM EventTarget
addEventListener fans events to many handlers

RxJS
Observable / Subject push values to subscribers

Vue · MobX
reactivity tracks deps and re-runs observers

<> Observer → subject.subscribe() · subject.setState() → notify() ⇒ each observer.update().

GoF intent: “Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.”

↓ DIVE DEEPER

Park walk

DIVE 03 · CONNECTIONS

▼ 100 m

Observers subscribe
attach to subject

→

subject changes state
setState()

→

notify() loops list
fan-out begins

→

✓ each update() runs
observers in sync

WHY IT FOLLOWS

→ Subject holds a list ⇒ it can broadcast to N observers it never had to name

→ Observers share update() ⇒ the subject talks to an interface, not concrete types

→ Subscribe at runtime ⇒ listeners come and go; the dependency graph is dynamic

→ Mediator centralizes complex notify logic · Singleton may hold the subject · contrast with Pub/Sub, which decouples both sides through a broker.

STRUCTURE · UML

Subject

- observers: Observer[]

+ attach(o: Observer): void

+ detach(o: Observer): void

+ notify(): void

notify() loops observers and calls update() «Observer»; concrete observers pull or receive state.

WHAT IT BUYS YOU

✓ Loose coupling — subject knows only the Observer interface, not concrete classes

✓ Broadcast communication — one notify() reaches every registered observer at once

✓ Dynamic relationships — observers attach and detach freely at runtime

✓ Open/closed extensibility — add new observer types without touching the subject

✓ Unexpected cascades — a blind notify can trigger wide, hard-to-trace update chains

↓ DIVE DEEPER

Microscope

DIVE 04 · DETAILS

▼ 1 cm

Definition: “Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified and updated automatically.”
— GoF, “Design Patterns”, 1994

THE CODE

```
type Observer = (data: number) => void;
class Subject {
  private obs = new Set<Observer>();
  subscribe(o: Observer) { this.obs.add(o); }
  unsubscribe(o: Observer) {
    this.obs.delete(o); // avoid leaks
  }
  notify(d: number) {
    this.obs.forEach((o) => o(d));
  }
}
```

VARIANTS

Push model
subject sends the changed data with the notification

EventEmitter
named events with multiple typed listeners (Node)

Fine-grained signals
auto-tracked deps re-run only what changed

Pull model
observers query the subject for what they need

RxJS streams
composable async pipelines with operators

Weak-ref listeners
GC can collect observers to avoid leaks

COMPARE THE ALTERNATIVES

Property	Observer	Pub/Sub	EventEmitter	RxJS
Subject knows observers	✓	✗	~	~
Broker / channel decoupling	✗	✓	~	~
Composable operators	✗	✗	✗	✓
Built-in unsubscribe	~	~	✓	✓
Async / backpressure	~	~	~	✓

Best fit → Observer: in-process deps · Pub/Sub: decoupled modules · EventEmitter: Node events · RxJS: async streams. (“~” = partial.)

INTERVIEW PITFALLS

Q “Observer vs Pub/Sub?”
Observer's subject holds direct references to its observers; Pub/Sub routes through a broker/channel so publisher and subscriber never know each other.

Q “Push or pull?”
Push sends the changed data in the notification; pull lets observers query what they need. Push is simpler, pull is more flexible.

Q “Most common bug?”
The lapsed-listener leak — observers that never unsubscribe keep the subject (and themselves) alive. Always pair subscribe with teardown.

Q “What if an observer throws?”
In a naive loop it aborts the rest. Wrap each callback, or deliver via a queue, so one failure can't starve other observers.

NUANCES & GOTCHAS

• Lapsed-listener leak — observers that forget to unsubscribe keep the subject alive and leak memory; pair subscribe with teardown

• Notification ordering — observers shouldn't assume an order; relying on it makes behavior fragile and registration-dependent

• Re-entrancy & cascades — an update that mutates the subject can re-trigger notify and loop or fire mid-iteration

• Sync vs async delivery — synchronous notify blocks the subject; async decouples but reorders and complicates error handling

• One observer throws — an unguarded loop lets a single failing observer skip the rest; isolate each callback

• Glitches / diamond — in reactive graphs a shared dep can fire observers twice; signal libs batch to avoid this

WATCH OUT

 **Anti-pattern:** Avoid it when there's a single known dependent (call it directly), when ordering or transactional consistency across listeners matters, or when implicit cascades make the flow impossible to follow — reach for explicit calls or a Mediator.

 **Modern take:** Modern UIs lean on fine-grained reactive signals (SolidJS, Vue, Angular) where dependencies are auto-tracked and updates batched, and on RxJS for async streams — Observer with the wiring and glitch-handling done for you.

• Vasyi Krupka · Senior Fullstack Engineer · learn-by-diving series

Source: GoF “Design Patterns” (1994) **v1**